

# **GeoSense AI**

## **API Contract Specification**

Complete Backend & AI Engine Interface Documentation

Version 1.0 | May 2026  
AYAIR 2026 — Innovation in Construction

# Table of Contents

- 1. Unified Response Structure
- 2. Client-Facing API (.NET Backend)
  - 2.1 Parcel Management
  - 2.2 Module 1 — AI Topographic Profile
  - 2.3 Module 2 — AI Soil Composition Estimate
  - 2.4 Module 3 — Bearing Capacity Estimate
  - 2.5 Module 4 — Construction Risk Score
  - 2.6 Module 5 — Optimized Borehole Campaign Plan
  - 2.7 Module 6 — Downloadable PDF Report
  - 2.8 Job Status & Polling
  - 2.9 Real-time Notifications (SignalR)
- 3. Internal AI Engine API (Python FastAPI)
  - 3.1 Topography Processing
  - 3.2 Soil Processing
  - 3.3 Risk Processing
  - 3.4 Borehole Optimization
  - 3.5 PDF Assembly
- 4. Error Codes Reference
- 5. Data Flow Summary
- Appendix A: HTTP Status Codes
- Appendix B: Data Type Definitions

# 1. Unified Response Structure

Every API response — whether from the client-facing .NET backend or the internal Python FastAPI — must follow this standardized envelope format. This ensures consistent error handling, predictable parsing, and unified logging across all layers of the GeoSense AI platform.

## 1.1 Success Response

```
{
  "statusCode": 200,
  "message": "Operation completed successfully",
  "errors": null,
  "data": { ... }
}
```

## 1.2 Error Response

```
{
  "statusCode": 400,
  "message": "Request validation failed",
  "errors": [
    {
      "field": "geoJson",
      "code": "INVALID_GEOMETRY",
      "message": "Polygon must contain at least 4 coordinates"
    }
  ],
  "data": null
}
```

## 1.3 Async Job Accepted Response

```
{
  "statusCode": 202,
  "message": "Job accepted for processing",
  "errors": null,
  "data": {
    "jobId": "550e8400-e29b-41d4-a716-446655440000",
    "parcelId": "parcel_12345",
    "status": "queued",
    "estimatedDuration": "2-6 hours"
  }
}
```

## 1.4 Response Field Definitions

| Field      | Type        | Required | Description                                        |
|------------|-------------|----------|----------------------------------------------------|
| statusCode | integer     | Yes      | HTTP status code (200, 201, 202, 400, 404, etc.)   |
| message    | string      | Yes      | Human-readable description of the result           |
| errors     | array null  | Yes      | Array of error objects on failure; null on success |
| data       | object null | Yes      | Payload object on success; null on failure         |

## 1.5 Error Object Structure

| Field   | Type        | Description                                           |
|---------|-------------|-------------------------------------------------------|
| field   | string null | Request field that caused the error (if applicable)   |
| code    | string      | Machine-readable error code for programmatic handling |
| message | string      | Detailed human-readable error description             |

## 2. Client-Facing API (.NET Backend)

These endpoints are exposed directly to the Angular frontend application. All requests require JWT Bearer authentication and return unified response envelopes.

### 2.1 Parcel Management

#### 2.1.1 POST /api/parcels — Register New Parcel

Description: Register a new land parcel boundary for analysis.

Request Headers:

| Header        | Value              | Required |
|---------------|--------------------|----------|
| Authorization | Bearer {JWT_TOKEN} | Yes      |
| Content-Type  | application/json   | Yes      |

Request Payload:

```
{
  "clientName": "Talaat Moustafa Group",
  "geoJson": {
    "type": "Polygon",
    "coordinates": [
      [ [30.0, 31.0], [30.1, 31.0], [30.1, 31.1], [30.0, 31.1], [30.0, 31.0] ]
    ]
  },
  "area": 50000,
  "areaUnit": "m2"
}
```

Validation Rules:

| Field      | Rule                                          | Error Code          |
|------------|-----------------------------------------------|---------------------|
| area       | Minimum 50,000 m <sup>2</sup> (5 hectares)    | PARCEL_TOO_SMALL    |
| geoJson    | Valid Polygon geometry, no self-intersections | INVALID_GEOMETRY    |
| clientName | Required, maximum 200 characters              | INVALID_CLIENT_NAME |
| areaUnit   | Enum: m2, hectares, acres                     | INVALID_UNIT        |

Response (201 Created):

```
{
  "statusCode": 201,
  "message": "Parcel created successfully",
  "errors": null,
  "data": {
    "parcelId": "parcel_550e8400",
    "boundingBox": {
      "minX": 30.0, "minY": 31.0,
      "maxX": 30.1, "maxY": 31.1
    },
    "area": 50000,
  }
}
```

```
    "createdAt": "2026-05-25T01:38:00Z"
  }
}
```

### 2.1.2 GET /api/parcels/{parcelId} — Retrieve Parcel

Path Parameters:

| Parameter | Type          | Description                     |
|-----------|---------------|---------------------------------|
| parcelId  | string (UUID) | Unique identifier of the parcel |

Response (200):

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "parcelId": "parcel_550e8400",
    "clientName": "Talaat Moustafa Group",
    "area": 50000,
    "status": "completed",
    "modulesCompleted": ["topography", "soil", "risk"],
    "boundingBox": { "minX": 30.0, "minY": 31.0, "maxX": 30.1, "maxY": 31.1 },
    "createdAt": "2026-05-25T01:38:00Z",
    "completedAt": "2026-05-25T04:15:00Z"
  }
}
```

## 2.2 Module 1 — AI Topographic Profile

### 2.2.1 POST /api/topography/jobs — Submit Topography Job

Description: Submit a topography analysis job for elevation mapping, slope analysis, contour generation, cut/fill calculation, and ponding risk assessment.

Request Payload:

```
{
  "parcelId": "parcel_550e8400",
  "options": {
    "contourInterval": 0.5,
    "slopeCategories": [2, 5, 15],
    "generateCutFill": true,
    "referencePlane": "auto"
  }
}
```

Options Field Definitions:

| Field           | Type     | Default    | Description                                              |
|-----------------|----------|------------|----------------------------------------------------------|
| contourInterval | number   | 0.5        | Elevation contour line interval in meters                |
| slopeCategories | number[] | [2, 5, 15] | Slope classification thresholds in percent               |
| generateCutFill | boolean  | true       | Enable cut-and-fill volume calculation                   |
| referencePlane  | string   | auto       | Cut/fill reference plane: auto or user-defined elevation |

Response (202 Accepted):

```
{
  "statusCode": 202,
  "message": "Topography job queued",
  "errors": null,
  "data": {
    "jobId": "job_topo_abc123",
    "parcelId": "parcel_550e8400",
    "status": "queued",
    "estimatedDuration": "2-6 hours"
  }
}
```

### 2.2.2 GET /api/topography/{parcelId} — Retrieve Topography Results

Path Parameters:

| Parameter | Type   | Description |
|-----------|--------|-------------|
| parcelId  | string | Parcel UUID |

Query Parameters:

| Parameter | Type | Required | Default | Description |
|-----------|------|----------|---------|-------------|
|-----------|------|----------|---------|-------------|

|              |         |    |      |                                         |
|--------------|---------|----|------|-----------------------------------------|
| includeTiles | boolean | No | true | Include S3 signed URLs for raster tiles |
| format       | string  | No | json | Enum: json, geojson                     |

Response (200):

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "parcelId": "parcel_550e8400",
    "elevation": {
      "min": 12.5,
      "max": 45.2,
      "mean": 28.7,
      "unit": "m"
    },
    "slopeAnalysis": {
      "distribution": [
        { "category": "flat", "range": "<2%", "percentage": 45, "color": "#4CAF50" },
        { "category": "gentle", "range": "2-5%", "percentage": 30, "color": "#8BC34A" },
        { "category": "moderate", "range": "5-15%", "percentage": 20, "color": "#FFC107" },
        { "category": "steep", "range": ">15%", "percentage": 5, "color": "#F44336" }
      ]
    },
    "cutFill": {
      "cutVolume": 4500,
      "fillVolume": 3200,
      "netVolume": -1300,
      "unit": "m³"
    },
    "contourLines": {
      "geoJsonUrl": "https://s3.amazonaws.com/.../contours.geojson?signature=...",
      "interval": 0.5
    },
    "pondingRisk": {
      "zonesCount": 3,
      "totalArea": 12500,
      "unit": "m²",
      "geoJsonUrl": "https://s3.amazonaws.com/.../ponding.geojson?signature=..."
    },
    "rasterTiles": {
      "elevation": "https://s3.amazonaws.com/.../elevation/{z}/{x}/{y}.png?signature=...",
      "slope": "https://s3.amazonaws.com/.../slope/{z}/{x}/{y}.png?signature=..."
    },
    "generatedAt": "2026-05-25T04:15:00Z"
  }
}
```

## 2.3 Module 2 — AI Soil Composition Estimate

### 2.3.1 POST /api/soil/jobs — Submit Soil Analysis Job

```
{
  "parcelId": "parcel_550e8400",
  "options": {
    "depths": ["0-20cm", "20-50cm", "50-100cm", "100-200cm"],
    "includeHeatmap": true
  }
}
```

### 2.3.2 GET /api/soil/{parcelId} — Retrieve Soil Results

Query Parameters:

| Parameter | Type   | Required | Default | Description                                |
|-----------|--------|----------|---------|--------------------------------------------|
| depth     | string | No       | 0-20cm  | Enum: 0-20cm, 20-50cm, 50-100cm, 100-200cm |

Response (200):

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "parcelId": "parcel_550e8400",
    "depth": "0-20cm",
    "composition": {
      "sand": 60,
      "silt": 25,
      "clay": 15,
      "unit": "%"
    },
    "properties": {
      "bulkDensity": 1.4,
      "bulkDensityUnit": "g/cm³",
      "organicCarbon": 1.2,
      "organicCarbonUnit": "%",
      "ph": 8.1
    },
    "classification": {
      "primaryType": "Sandy Loam",
      "usdaClass": "Loamy",
      "aiConfidence": 0.85
    },
    "multiDepthProfile": [
      { "depth": "0-20cm", "sand": 60, "clay": 15, "type": "Sandy Loam" },
      { "depth": "20-50cm", "sand": 50, "clay": 20, "type": "Loam" },
      { "depth": "50-100cm", "sand": 40, "clay": 25, "type": "Clay Loam" },
      { "depth": "100-200cm", "sand": 30, "clay": 35, "type": "Clay" }
    ],
    "heatmapTileUrl": "https://s3.amazonaws.com/.../soil_heatmap/{z}/{x}/{y}.png?signature=...",
    "generatedAt": "2026-05-25T04:15:00Z"
  }
}
```

## 2.4 Module 3 — Bearing Capacity Estimate

### 2.4.1 POST /api/bearing/jobs — Submit Bearing Capacity Job

```
{
  "parcelId": "parcel_550e8400",
  "foundationType": "shallow"
}
```

*foundationType Enum: shallow | deep | pile | mat*

### 2.4.2 GET /api/bearing/{parcelId} — Retrieve Bearing Capacity Results

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "parcelId": "parcel_550e8400",
    "bearingCapacity": {
      "value": 150,
      "unit": "kPa",
      "category": "Medium",
      "range": "75-200 kPa",
      "trafficLight": "yellow"
    },
    "foundationRecommendation": {
      "type": "Shallow Foundations",
      "suitable": true,
      "maxFloorsWithoutDeepFoundation": 5,
      "floorCountCategory": "3-5 floors"
    },
    "soilFactors": {
      "clayContent": 15,
      "sandContent": 60,
      "moistureIndex": 0.32,
      "depthToWaterTable": 8.5
    },
    "disclaimer": "This is an AI estimate for preliminary planning only. Physical borehole verification",
    "generatedAt": "2026-05-25T04:15:00Z"
  }
}
```

Bearing Capacity Categories:

| Category | Range (kPa) | Traffic Light | Floor Suitability |
|----------|-------------|---------------|-------------------|
| Low      | < 75        | Red           | 1-2 floors only   |
| Medium   | 75 - 200    | Yellow        | 3-5 floors        |
| High     | > 200       | Green         | 6-10+ floors      |

## 2.5 Module 4 — Construction Risk Score

### 2.5.1 POST /api/risks/jobs — Submit Risk Assessment Job

```
{
  "parcelId": "parcel_550e8400",
  "riskTypes": ["flood", "seismic", "expansiveSoil", "liquefaction"]
}
```

### 2.5.2 GET /api/risks/{parcelId} — Retrieve Risk Results

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "parcelId": "parcel_550e8400",
    "overallScore": 42,
    "overallRiskLevel": "Moderate",
    "maxScore": 100,
    "riskBreakdown": {
      "flood": {
        "score": 65,
        "level": "High",
        "factors": ["Low slope <2%", "High TWI index", "Proximity to drainage basin"],
        "geoJsonUrl": "https://s3.amazonaws.com/.../flood_zones.geojson?signature=..."
      },
      "seismic": {
        "score": 20,
        "level": "Low",
        "factors": ["NCSR zone classification: Low", "Far from active fault lines"],
        "source": "USGS/Egypt NCSR"
      },
      "expansiveSoil": {
        "score": 45,
        "level": "Moderate",
        "factors": ["Clay content: 15%", "Shrink-swell potential: Medium"],
        "replacementDepth": 1.5
      },
      "liquefaction": {
        "score": 30,
        "level": "Low-Moderate",
        "factors": ["Sandy soil present", "Water table depth: 8.5m"],
        "susceptibility": "Low"
      }
    },
    "generatedAt": "2026-05-25T04:15:00Z"
  }
}
```

Risk Score Interpretation:

| Score Range | Level     | Color      | Interpretation                                |
|-------------|-----------|------------|-----------------------------------------------|
| 0-20        | Very Low  | Dark Green | Minimal risk, standard construction           |
| 21-40       | Low       | Green      | Low risk, minor precautions                   |
| 41-60       | Moderate  | Yellow     | Moderate risk, engineering review required    |
| 61-80       | High      | Orange     | High risk, detailed investigation mandatory   |
| 81-100      | Very High | Red        | Very high risk, significant mitigation needed |

## 2.6 Module 5 — Optimized Borehole Campaign Plan

### 2.6.1 POST /api/boreholes/jobs — Submit Borehole Optimization Job

```
{
  "parcelId": "parcel_550e8400",
  "parameters": {
    "maxSpacing": 30,
    "minBoreholes": 12,
    "targetDepth": 20,
    "unit": "m"
  }
}
```

### 2.6.2 GET /api/boreholes/{parcelId} — Retrieve Borehole Plan

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "parcelId": "parcel_550e8400",
    "recommendation": {
      "minimumRequired": 12,
      "optimalCount": 18,
      "coveragePercentage": 85,
      "gridSize": "30m spacing"
    },
    "placement": {
      "strategy": "Adaptive grid with hotspots",
      "points": [
        {
          "id": "BH-001",
          "latitude": 31.05,
          "longitude": 30.02,
          "priority": "High",
          "reason": "Soil variability hotspot",
          "estimatedDepth": 20
        }
      ]
    },
    "geoJsonUrl": "https://s3.amazonaws.com/.../boreholes.geojson?signature=..."
  },
  "costAnalysis": {
    "traditionalApproach": {
      "boreholes": 30,
      "estimatedCost": 420000,
      "currency": "EGP"
    },
    "optimizedApproach": {
      "boreholes": 12,
      "estimatedCost": 180000,
      "currency": "EGP"
    },
    "savings": {
      "amount": 240000,
      "currency": "EGP",
      "percentage": 57
    }
  },
  "generatedAt": "2026-05-25T04:15:00Z"
}
```

Placement Point Priority Levels:

| Priority | Description                              | Action Required    |
|----------|------------------------------------------|--------------------|
| Critical | Extreme soil variability or known hazard | Mandatory borehole |

|        |                                       |                      |
|--------|---------------------------------------|----------------------|
| High   | Significant soil variability detected | Strongly recommended |
| Medium | Moderate variability, boundary zone   | Recommended          |
| Low    | Homogeneous area, confirmation point  | Optional             |

## 2.7 Module 6 — Downloadable PDF Report

### 2.7.1 POST /api/reports/jobs — Submit PDF Generation Job

```
{
  "parcelId": "parcel_550e8400",
  "reportOptions": {
    "language": "en",
    "includeMaps": true,
    "includeTables": true,
    "includeRiskMatrix": true,
    "disclaimerLevel": "full",
    "companyName": "Talaat Moustafa Group",
    "projectName": "New Alamein Phase 3"
  }
}
```

Report Options Definitions:

| Field             | Type    | Default | Description                           |
|-------------------|---------|---------|---------------------------------------|
| language          | string  | en      | Report language: en, ar               |
| includeMaps       | boolean | true    | Include interactive map screenshots   |
| includeTables     | boolean | true    | Include data tables                   |
| includeRiskMatrix | boolean | true    | Include risk assessment matrix        |
| disclaimerLevel   | string  | full    | Enum: full, standard, minimal         |
| companyName       | string  | null    | Client company name for report header |
| projectName       | string  | null    | Project name for report header        |

### 2.7.2 GET /api/reports/{parcelId}/pdf — Download PDF Report

Path Parameters:

| Parameter | Type   | Description |
|-----------|--------|-------------|
| parcelId  | string | Parcel UUID |

Query Parameters:

| Parameter | Type    | Required | Default | Description                   |
|-----------|---------|----------|---------|-------------------------------|
| download  | boolean | No       | true    | Force download vs inline view |
| version   | string  | No       | latest  | Report version timestamp      |

Response Headers:

Content-Type: application/pdf

Content-Disposition: attachment; filename="GeoSense\_Report\_parcel\_550e8400\_20260525.pdf"  
Content-Length: 2450000

Response Body: Binary PDF stream (4-6 pages)

Error Response (404):

```
{
  "statusCode": 404,
  "message": "Report not found or still generating",
  "errors": [
    { "code": "REPORT_NOT_READY", "message": "PDF generation is still in progress. Please check job status" },
  ],
  "data": null
}
```

## 2.8 Job Status & Polling

### 2.8.1 GET /api/jobs/{jobId}/status — Check Job Status

Description: Check the processing status of any asynchronous job across all modules.

Path Parameters:

| Parameter | Type          | Description                                              |
|-----------|---------------|----------------------------------------------------------|
| jobId     | string (UUID) | Unique job identifier returned from POST /jobs endpoints |

Response (200):

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "jobId": "job_topo_abc123",
    "parcelId": "parcel_550e8400",
    "module": "topography",
    "status": "completed",
    "progressPercentage": 100,
    "startedAt": "2026-05-25T01:40:00Z",
    "completedAt": "2026-05-25T04:15:00Z",
    "nextModule": "soil",
    "message": "Topography analysis completed successfully"
  }
}
```

Status Enum Values:

| Status     | Description               | Frontend Action                  |
|------------|---------------------------|----------------------------------|
| queued     | Job is waiting in queue   | Show 'Queued' progress indicator |
| processing | Job is actively running   | Show animated progress bar       |
| completed  | Job finished successfully | Enable 'View Results' button     |
| failed     | Job encountered an error  | Show error details, offer retry  |

### 2.8.2 Polling Strategy

The Angular frontend should implement the following polling strategy for job status monitoring:

- Initial poll: 5 seconds after job submission
- Subsequent polls: Every 5 seconds for first 2 minutes
- Extended polling: Every 30 seconds after 2 minutes
- Maximum polling duration: 6 hours (job timeout)
- Fallback: Switch to WebSocket (SignalR) when available

## 2.9 Real-time Notifications (SignalR)

Hub Endpoint: /hubs/job-status

Protocol: WebSockets with automatic fallback to Server-Sent Events

### Event: ModuleCompleted

```
{
  "module": "topography",
  "parcelId": "parcel_550e8400",
  "jobId": "job_topo_abc123",
  "timestamp": "2026-05-25T04:15:00Z",
  "message": "Topography module completed"
}
```

SignalR Connection Lifecycle:

| Phase      | Action                               | Trigger                       |
|------------|--------------------------------------|-------------------------------|
| Connect    | Initialize connection on app startup | User opens dashboard          |
| Join Group | Subscribe to parcel-specific group   | User submits job              |
| Listen     | Handle ModuleCompleted events        | Backend broadcasts completion |
| Disconnect | Clean up on app close/logout         | User navigates away           |

## 3. Internal AI Engine API (Python FastAPI)

These endpoints are called exclusively by the .NET Hangfire background workers. They are not exposed to the frontend directly. All responses follow the same unified envelope format.

### 3.1 Topography Processing

#### 3.1.1 POST /api/v1/topography/jobs

*Called by: .NET Hangfire ProcessTopographyJob*

Request Payload:

```
{
  "jobId": "job_topo_abc123",
  "parcelId": "parcel_550e8400",
  "bbox": {
    "minX": 29.95,
    "minY": 30.95,
    "maxX": 30.15,
    "maxY": 31.15
  },
  "geoJson": {
    "type": "Polygon",
    "coordinates": [ [ [30.0, 31.0], [30.1, 31.0], [30.1, 31.1], [30.0, 31.1], [30.0, 31.0] ] ]
  },
  "options": {
    "contourInterval": 0.5,
    "slopeCategories": [2, 5, 15],
    "generateCutFill": true,
    "referencePlane": "auto"
  }
}
```

Response (202):

```
{
  "statusCode": 202,
  "message": "Python topography job queued",
  "errors": null,
  "data": {
    "pythonJobId": "pyjob_topo_xyz789",
    "status": "queued",
    "acceptedAt": "2026-05-25T01:40:00Z"
  }
}
```

#### 3.1.2 GET /api/v1/topography/jobs/{pythonJobId}

*Called by: .NET Hangfire CheckPythonStatusJob (every 30 seconds)*

Response (200) — Processing:

```
{
  "statusCode": 200,
  "message": "Job in progress",
  "errors": null,
  "data": {
    "pythonJobId": "pyjob_topo_xyz789",
    "status": "processing",
    "progressPercentage": 65,
    "currentStage": "Slope Classification",
    "stageDetails": "Classifying pixels into slope categories"
  }
}
```

Response (200) — Completed:

```

{
  "statusCode": 200,
  "message": "Job completed",
  "errors": null,
  "data": {
    "pythonJobId": "pyjob_topo_xyz789",
    "status": "completed",
    "results": {
      "elevation": { "min": 12.5, "max": 45.2, "mean": 28.7 },
      "slopeDistribution": [
        { "category": "flat", "percentage": 45 },
        { "category": "gentle", "percentage": 30 },
        { "category": "moderate", "percentage": 20 },
        { "category": "steep", "percentage": 5 }
      ],
      "cutFill": { "cut": 4500, "fill": 3200, "net": -1300 },
      "tileUrls": {
        "elevation": "s3://bucket/elevation/{z}/{x}/{y}.png",
        "slope": "s3://bucket/slope/{z}/{x}/{y}.png"
      },
      "geoJsonUrls": {
        "contours": "s3://bucket/contours.geojson",
        "ponding": "s3://bucket/ponding.geojson"
      },
      "metadata": {
        "copernicusDemVersion": "COP-30",
        "processingTime": 7200,
        "pixelResolution": 30
      }
    },
    "completedAt": "2026-05-25T04:15:00Z"
  }
}

```

## 3.2 Soil Processing

### 3.2.1 POST /api/v1/soil/jobs

```
{
  "jobId": "job_soil_def456",
  "parcelId": "parcel_550e8400",
  "bbox": { "minX": 29.95, "minY": 30.95, "maxX": 30.15, "maxY": 31.15 },
  "geoJson": { ... },
  "depths": ["0-20cm", "20-50cm", "50-100cm", "100-200cm"]
}
```

### 3.2.2 GET /api/v1/soil/jobs/{pythonJobId}

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "pythonJobId": "pyjob_soil_abc111",
    "status": "completed",
    "results": {
      "surface": {
        "sand": 60, "silt": 25, "clay": 15,
        "bulkDensity": 1.4, "organicCarbon": 1.2, "ph": 8.1,
        "classification": "Sandy Loam"
      },
      "depthProfiles": [ ... ],
      "heatmapTileUrl": "s3://bucket/soil_heatmap/{z}/{x}/{y}.png",
      "dataSources": ["ISRIC SoilGrids v2.0", "AfSIS Africa Soil", "Sentinel-2 L2A"]
    }
  }
}
```

## 3.3 Risk Processing

### 3.3.1 POST /api/v1/risks/jobs

```
{
  "jobId": "job_risk_jkl012",
  "parcelId": "parcel_550e8400",
  "bbox": { ... },
  "geoJson": { ... },
  "riskTypes": ["flood", "seismic", "expansiveSoil", "liquefaction"],
  "soilData": {
    "clayContent": 15,
    "sandContent": 60,
    "waterTableDepth": 8.5
  }
}
```

### 3.3.2 GET /api/v1/risks/jobs/{pythonJobId}

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "pythonJobId": "pyjob_risk_abc222",
    "status": "completed",
    "results": {
      "overallScore": 42,
      "flood": { "score": 65, "level": "High", "zonesGeoJson": "s3://.../flood.geojson" },
      "seismic": { "score": 20, "level": "Low", "zone": "NCSR-Low" },
      "expansiveSoil": { "score": 45, "level": "Moderate" },
      "liquefaction": { "score": 30, "level": "Low" }
    }
  }
}
```

## 3.4 Borehole Optimization

### 3.4.1 POST /api/v1/boreholes/jobs

```
{
  "jobId": "job_bore_mno345",
  "parcelId": "parcel_550e8400",
  "bbox": { ... },
  "geoJson": { ... },
  "soilVariability": {
    "hotspotZones": [ ... ],
    "homogeneousZones": [ ... ]
  },
  "parameters": {
    "maxSpacing": 30,
    "minBoreholes": 12,
    "targetDepth": 20
  }
}
```

### 3.4.2 GET /api/v1/boreholes/jobs/{pythonJobId}

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "pythonJobId": "pyjob_bore_abc333",
    "status": "completed",
    "results": {
      "minimumRequired": 12,
      "optimalCount": 18,
      "placementPoints": [
        { "id": "BH-001", "lat": 31.05, "lng": 30.02, "priority": "High", "reason": "Soil hotspot" }
      ],
      "costComparison": {
        "traditional": { "count": 30, "cost": 420000 },
        "optimized": { "count": 12, "cost": 180000 },
        "savings": { "amount": 240000, "percentage": 57 }
      }
    }
  }
}
```

## 3.5 PDF Assembly (Optional Python Endpoint)

### 3.5.1 POST /api/v1/reports/jobs

```
{
  "jobId": "job_pdf_pqr678",
  "parcelId": "parcel_550e8400",
  "moduleResults": {
    "topography": { ... },
    "soil": { ... },
    "bearing": { ... },
    "risk": { ... },
    "borehole": { ... }
  },
  "reportOptions": {
    "companyName": "Talaat Moustafa Group",
    "projectName": "New Alamein Phase 3",
    "language": "en"
  }
}
```

### 3.5.2 GET /api/v1/reports/jobs/{pythonJobId}

```
{
  "statusCode": 200,
  "message": "Success",
  "errors": null,
  "data": {
    "pythonJobId": "pyjob_pdf_abc444",
    "status": "completed",
    "results": {
      "pdfUrl": "s3://bucket/reports/report_550e8400_20260525.pdf",
      "pageCount": 6,
      "sizeBytes": 2450000
    }
  }
}
```

## 4. Error Codes Reference

The following error codes are used consistently across both the .NET backend and Python FastAPI layers. Each code maps to a specific HTTP status code and has a defined resolution strategy.

| Error Code               | HTTP Status | Description                                      | Resolution                                                              |
|--------------------------|-------------|--------------------------------------------------|-------------------------------------------------------------------------|
| INVALID_GEOMETRY         | 400         | GeoJSON polygon is invalid or self-intersecting  | Validate geometry using NetTopologySuite before saving                  |
| PARCEL_TOO_SMALL         | 400         | Parcel area is below 5 hectares (50,000 m²)      | Select a larger land parcel or contact support for minimum requirements |
| INVALID_CLIENT_NAME      | 400         | Client name is missing or exceeds 200 characters | Provide a valid client name within character limits                     |
| INVALID_UNIT             | 400         | Area unit is not in allowed enum                 | Use one of: m2, hectares, acres                                         |
| PARCEL_NOT_FOUND         | 404         | Parcel ID does not exist in database             | Verify parcelId or create new parcel                                    |
| JOB_NOT_FOUND            | 404         | Job ID does not exist in queue or database       | Verify jobId or resubmit job                                            |
| JOB_NOT_COMPLETED        | 409         | Results requested before job completion          | Poll status endpoint until status=completed                             |
| MODULE_NOT_AVAILABLE     | 422         | Requested module not yet implemented             | Check API version or contact support                                    |
| GEE_RATE_LIMIT           | 429         | Google Earth Engine API quota exceeded           | Implement exponential backoff; retry after 60 seconds                   |
| PROCESSING_TIMEOUT       | 504         | Python AI Engine processing exceeded timeout     | Job will be marked failed; retry or contact support                     |
| INTERNAL_ERROR           | 500         | Unhandled server error                           | Log incident ID; contact technical support                              |
| REPORT_NOT_READY         | 404         | PDF generation is still in progress              | Wait for ModuleCompleted event or poll job status                       |
| INVALID_TOKEN            | 401         | JWT token is missing, expired, or invalid        | Refresh authentication token                                            |
| INSUFFICIENT_PERMISSIONS | 403         | User lacks permission for this parcel            | Contact account administrator                                           |

## 5. Data Flow Summary

The following sequence describes the complete data flow from user action to final report delivery, illustrating how the client-facing API, background workers, and AI engine interact.

### 1. User Action — Frontend (Angular)

User draws polygon on MapLibre GL JS map or uploads GeoJSON file

### 2. Parcel Registration — POST /api/parcels

Angular sends GeoJSON + area to .NET API; receives 201 Created + parcelId

### 3. Job Submission — POST /api/{module}/jobs

User clicks 'Analyze'; Angular sends job request; receives 202 Accepted + jobId

### 4. Queue Enqueue — .NET Hangfire

Controller enqueues Hangfire background job; returns immediately

### 5. Worker Pickup — Hangfire Server

Worker picks up job from queue; reads parcel from PostGIS

### 6. Python Dispatch — POST /api/v1/{module}/jobs

Hangfire calls FastAPI with bbox + geoJson; receives pythonJobId

### 7. Celery Processing — Python FastAPI + Celery

FastAPI enqueues Celery task; worker processes via GEE/GDAL/ML

### 8. Status Polling — GET /api/v1/{module}/jobs/{id}

Hangfire polls Python every 30s until status=completed

### 9. Result Storage — .NET Hangfire

On completion: parse JSON → save to PostGIS → cache in Redis (TTL 1hr)

### 10. Real-time Notify — SignalR Hub

Broadcast ModuleCompleted event to Angular clients in parcel group

### 11. Frontend Fetch — GET /api/{module}/{parcelId}

Angular fetches results from .NET API (Redis cache → fast)

### 12. Map Rendering — MapLibre GL JS

Angular renders raster tiles (S3 signed URLs) + GeoJSON overlays

### **13. Dashboard Display — Angular + ngx-charts**

Stats cards, pie charts, gauge charts, traffic lights rendered

### **14. PDF Download — GET /api/reports/{id}/pdf**

User clicks Download; .NET generates PDF via QuestPDF; streams bytes

### **15. Cleanup — Celery Beat**

After 24h: archive old jobs; delete temp GeoTIFFs; keep S3 artifacts 30 days

## Appendix A: HTTP Status Codes

| Status Code | Name                  | Usage in GeoSense AI                                            |
|-------------|-----------------------|-----------------------------------------------------------------|
| 200         | OK                    | Successful synchronous request (GET parcel, GET results)        |
| 201         | Created               | New resource created successfully (POST /api/parcels)           |
| 202         | Accepted              | Async job queued successfully (all POST /jobs endpoints)        |
| 400         | Bad Request           | Request validation failed (invalid geometry, missing fields)    |
| 401         | Unauthorized          | JWT token missing, expired, or invalid                          |
| 403         | Forbidden             | User lacks permission for requested resource                    |
| 404         | Not Found             | Resource does not exist (parcel, job, report)                   |
| 409         | Conflict              | Resource in invalid state (results requested before completion) |
| 422         | Unprocessable Entity  | Semantic validation failed (module not available)               |
| 429         | Too Many Requests     | Rate limit exceeded (GEE API quota, client throttling)          |
| 500         | Internal Server Error | Unhandled server exception                                      |
| 502         | Bad Gateway           | Python FastAPI service unavailable                              |
| 503         | Service Unavailable   | Service temporarily down for maintenance                        |
| 504         | Gateway Timeout       | Python processing exceeded maximum duration                     |

# Appendix B: Data Type Definitions

## B.1 BoundingBox

```
{
  "minX": 29.95,    // float, WGS84 longitude
  "minY": 30.95,    // float, WGS84 latitude
  "maxX": 30.15,    // float, WGS84 longitude
  "maxY": 31.15     // float, WGS84 latitude
}
```

## B.2 GeoJSON Polygon

```
{
  "type": "Polygon",
  "coordinates": [
    [
      [30.0, 31.0], // [longitude, latitude]
      [30.1, 31.0],
      [30.1, 31.1],
      [30.0, 31.1],
      [30.0, 31.0] // Must close the ring
    ]
  ]
}
```

## B.3 Slope Category

| Field      | Type   | Description                                        |
|------------|--------|----------------------------------------------------|
| category   | string | Classification name: flat, gentle, moderate, steep |
| range      | string | Human-readable range: <2%, 2-5%, 5-15%, >15%       |
| percentage | number | Percentage of total parcel area (0-100)            |
| color      | string | Hex color code for map rendering                   |

## B.4 Raster Tile URL Pattern

```
https://{bucket}.s3.amazonaws.com/{parcelId}/{layer}/{z}/{x}/{y}.png?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-
// {layer} values: elevation, slope, soil_heatmap, risk_flood
// {z}/{x}/{y}: Standard XYZ tile coordinates
// Signature: Temporary S3 signed URL (expires in 1 hour)
```

## B.5 Job Status Enum

```
enum JobStatus {
  QUEUED      = "queued",    // Waiting in queue
  PROCESSING  = "processing", // Actively running
  COMPLETED  = "completed",  // Finished successfully
  FAILED      = "failed"     // Error occurred
}
```

## B.6 Risk Level Enum

```
enum RiskLevel {  
    VERY_LOW    = "Very Low",    // Score 0-20  
    LOW         = "Low",         // Score 21-40  
    MODERATE    = "Moderate",    // Score 41-60  
    HIGH        = "High",        // Score 61-80  
    VERY_HIGH   = "Very High"    // Score 81-100  
}
```

## B.7 Foundation Type Enum

```
enum FoundationType {  
    SHALLOW = "shallow", // Spread footings, strip foundations  
    DEEP     = "deep",    // Pile foundations, caissons  
    PILE     = "pile",    // Driven or bored piles  
    MAT      = "mat",     // Raft/mat foundation  
}
```

# Document Control

| Attribute      | Value                                            |
|----------------|--------------------------------------------------|
| Document Title | GeoSense AI — API Contract Specification         |
| Version        | 1.0                                              |
| Date           | May 25, 2026                                     |
| Author         | GeoSense AI Technical Team                       |
| Project        | AYAIR 2026 — Innovation in Construction          |
| Classification | Internal — Development Team                      |
| Review Cycle   | Bi-weekly during development phase               |
| Distribution   | Backend Team, Frontend Team, AI/ML Team, QA Team |

| Version | Date       | Changes                                                              | Author         |
|---------|------------|----------------------------------------------------------------------|----------------|
| 1.0     | 2026-05-25 | Initial release — all 6 modules, unified response, FastAPI internals | Technical Lead |